

Tourism Government — Ultimate Technical Interview Prep Handbook

Exhaustive, Deep Explanations for All 15 Microservices & Testing Questions

■ INTERVIEW STRATEGY WARNING:

This master prep guide translates all the hot-topic technical questions shared by your peers into structured, senior-level technical answers. It walks through Spring Boot, React connections, circuit breaker mechanics, and details the deep architectural breakdown of the **Eureka 5-year port-change production question**.

■ Explain your project (TourismGov).

Our project is a scalable, cloud-native **Government Tourism Management System** built using Spring Boot microservices and a React frontend. The application allows government administrators to register heritage sites, schedule cultural programs, coordinate public events, check regulatory compliances, and dynamically aggregate statistical reports while triggering real-time email alerts and notification records to tourists and officers.

■■ **Your Verbal Pitch:** "I developed a cloud-native Tourism Government Suite that utilizes a Database-per-Service model. We coordinate data dynamically across modules like Site Registry, Compliance, Notifications, and Reports to offer real-time analytics to admin officers."

■ What are the design patterns used in your project?

Our project leverages several microservice and software design patterns:

1. **Database-per-Service:** Each microservice manages its own isolated MySQL database schema (e.g. `notificationdb`, `reportdb`) to ensure data autonomy.
2. **Observer Pattern:** The Notification Service acts as an observer that listens for events (like site creation or compliance violations) and pushes notifications.
3. **Observer/Fallback Pattern:** OpenFeign fallbacks bound directly to remote clients protect our system from cascaded failures.
4. **API Gateway Pattern:** Acts as the single entry point for routing, authentication filters, and rate limiting.

■■ **Your Verbal Pitch:** "We used a Database-per-Service model to prevent database locking, coordinated inter-service events via a central Notification observer, and implemented Feign-level Fallback patterns for robust fault-tolerance."

■ What is the purpose of the service registry (Eureka)?

In a microservices ecosystem, service instances can scale up, down, or change IP addresses dynamically. A **Service Registry (Eureka)** acts as a central database where every active microservice registers its IP address and port upon startup. When a microservice needs to call another (e.g., `ReportService` calls `UserService`), it queries Eureka for the target service's current location, completely eliminating hardcoded IPs.

■■ **Your Verbal Pitch:** "Eureka acts as our dynamic microservice lookup directory, enabling client-side load balancing and auto-discovery without hardcoding IP addresses."

■ Eureka Server you are using in production, you change the port after 5 years. Will it make any difference in the application? (Explain deep)

■ YES, it will completely break the application IF you have hardcoded the Eureka service URL port in your client microservices' properties. However, ■ NO, it will NOT make any difference if you implement production best-practices like **DNS Load Balancing** or **Centralized Spring Cloud Config Server**.

■ Where it Breaks (The Local Hardcoded Mistake):

If your client microservices are configured locally like this:

```
eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka/
```

Changing Eureka's port after 5 years from `8761` to `9000` means every client will instantly lose connection. They will fail to discover each other, resulting in a ****catastrophic, cascading system-wide downtime****.

■ How We Prevent This in Production (No Difference / Seamless Port Change):

1. DNS Mapping / Load Balancer DNS (Best Practice):

In production, we never use static ports or localhost. We map the Eureka Service to a permanent Load Balancer URL, e.g.:

```
eureka.client.serviceUrl.defaultZone=http://eureka.tourismgov.internal/eureka/
```

We can change the underlying physical server ports as much as we want; we only need to update the Load Balancer target port. The client configurations remain 100% untouched and never break!

2. Spring Cloud Config Server & Dynamic Actuator Refresh:

If configurations are stored centrally in a Git repository, we can update the Eureka port in one single config file, trigger a `/actuator/refresh` request, and all microservices will dynamically fetch the new port without requiring a redeployment or server restart!

■ ■ **Your Verbal Pitch:** "Yes, it will crash the system if ports are hardcoded locally. However, in production we avoid this by pointing to a Load Balancer DNS Name (like `eureka.tourismgov.internal`) so that port changes remain completely invisible and seamless to our client microservices."

■ Explain how you connect backend and Frontend.

We connect our React frontend and Spring Boot backend via an ****API Gateway****. The React frontend makes HTTP requests using `Axios` to a single gateway port (e.g. `8080`). The Gateway intercepts the request, verifies the JWT security token, and routes the request dynamically to the corresponding microservice (e.g. `/api/notifications` to the Notification Service).

■ ■ **Your Verbal Pitch:** "Our React frontend communicates strictly with a single endpoint - our Spring Cloud API Gateway - using Axios. The Gateway handles routing, CORS, and JWT authentication filters."

■ Explain MVC architecture from Controller to Service.

MVC (Model-View-Controller) is a software architecture pattern:

1. ****Controller (Entry point):**** Receives HTTP requests, performs basic DTO validation, and calls the Service layer.
2. ****Service (Business Layer):**** Implements business algorithms, orchestrates calls across clients, and coordinates database saves.
3. ****Repository (Data Access):**** Interfaces directly with the database using Hibernate JPA queries.

■ ■ **Your Verbal Pitch:** "Our Controller receives requests, validates DTO annotations, calls the ServiceImpl where business rules reside, which in turn calls the repository layer to save data."

■ What are the validations you used? Explain the DTO layer.

We use Jakarta Bean Validation annotations directly inside our **DTO (Data Transfer Object)** layer to validate payloads before they touch our business logic. Examples include `@NotNull(message = "Scope is required")` and `@Email`. Inside controllers, we use the `@Valid` annotation to trigger these validation checks automatically.

■■ **Your Verbal Pitch:** "We place standard validations like `@NotNull` and `@Size` on our DTO fields. When our controller receives a request, the `@Valid` annotation automatically executes these validations."

■ Why do we prefer Constructor Injection instead of Field Injection?

We prefer **Constructor Injection** (using Lombok's `@RequiredArgsConstructor` on final fields) over Field Injection (`@Autowired` directly on fields) because:

1. **Null-Safety:** Fields are marked `final`, ensuring they are immutable and initialized at compilation time.
2. **Testing Simplicity:** It allows us to easily inject mocked dependencies in simple JUnit test classes without needing a Spring container.

■■ **Your Verbal Pitch:** "Constructor injection ensures our dependencies are final, immutable, and null-safe, making our classes extremely easy to test in isolation using simple JUnit constructors."

■ Explain the code of circuit breakers, all sliding windows, and configurations.

We configured a **Count-Based sliding window** of size `10` with a `50%` failure threshold. We also allowed exactly **5 trial requests** in the **HALF-OPEN** state to test if the service has recovered. At a 50% threshold, at least 3 out of 5 requests must succeed to close (repair) the circuit; otherwise, it trips back to OPEN.

■■ **Your Verbal Pitch:** "Our sliding window tracks the last 10 requests. If 5 or more fail, the circuit opens for 10 seconds. When transitioning to Half-Open, it runs 5 trials, requiring at least 3 successes to reset."

■ Have you written test cases? Explain JUnit testing using @Mock and @InjectMocks.

Yes, we wrote robust unit test suites for `NotificationService` and `ReportingService` using JUnit 5 and Mockito. We use `@Mock` to create simulated, lightweight fakes of downstream Feign Clients and SQL Repositories, and use `@InjectMocks` to instantiate our service class and automatically inject these mocks into it, allowing our tests to execute in milliseconds with zero active databases.

■■ **Your Verbal Pitch:** "Yes, we wrote complete Mockito unit tests. We mock out database and external client calls to isolate our service business logic, verifying happy paths, fallbacks, and security boundaries in milliseconds."